

Avaliando modelos de linguagem

Trabalhando com Texto em Python I — Unidade 4

Padrão-ouro, concordância entre anotadores, kappa de Cohen e métricas de desempenho (acurácia, precisão, revocação, F1).

Adaptado de *Applied Language Technology* (Universidade de Helsinque) — applied-language-technology.mooc.fi

Objetivos de aprendizagem

Ao final desta unidade, você deverá:

- entender o que é um **padrão-ouro** (*gold standard*);
- saber avaliar a **concordância** entre anotadores humanos;
- compreender **métricas simples** para avaliar o desempenho do PLN.

1. O que é um padrão-ouro (*gold standard*)?

Um **padrão-ouro** (*gold standard*) — também chamado de *ground truth* (“verdade fundamental”) — refere-se a **dados verificados por humanos** usados como referência (*benchmark*) para avaliar o desempenho de algoritmos. Em PLN, os padrões-ouro medem quão bem os **humanos** desempenham uma tarefa.

O objetivo do PLN é permitir que computadores **alcancem ou superem** o desempenho humano em uma tarefa pré-definida. Medir isso exige uma referência — e é o padrão-ouro que a fornece: um **ponto de referência**.

É importante entender que **padrões-ouro são abstrações do uso da língua**. As classes gramaticais, por exemplo, não são dadas pela natureza: representam uma abstração que impõe estrutura à língua. A língua é naturalmente **ambígua e subjetiva**, e as abstrações podem ser incompletas — não temos certeza de que todos classificariam as palavras da mesma forma. Por isso precisamos **medir a confiabilidade** de um padrão-ouro: até que ponto os humanos **concordam** na tarefa.

2. Medindo a confiabilidade manualmente

A **confiabilidade** — geralmente entendida como a **concordância entre múltiplos anotadores** — pode ser medida manualmente.

2.1 Passo 1 — Anotar os dados

A **análise de sentimento** consiste em determinar o sentimento de um texto. Treinar um modelo assim exige dados de treino: exemplos de textos associados a sentimentos. Classifique os *tweets* abaixo em **positivo**, **neutro** ou **negativo**. Anote sua decisão (uma por linha), sem discutir com o colega ao lado:

1. Updated: HSL GTFS (Helsinki, Finland) <https://t.co/fWEpzmNQLz>
2. current weather in Helsinki: broken clouds, -8 C, 100% humidity, wind 4kmh, pressure 1061mb

3. CNN: "WallStreetBets Redditors go ballistic over GameStop's sinking share price"
4. Baana bicycle counter. Today: 3 Same time last week: 1058 Trend: -99% This year: 819518 Last year: 802079 #Helsinki #cycling
5. Elon Musk is now tweeting about #bitcoin
6. A perfect Sunday walk in the woods just a few steps from home.
7. Went to Domino's today. It was so amazing and I think I got damn good dessert as well. . .
8. Choo Choo. There's our train! #holidayahead
9. Happy women's day, kisses to all you beautiful ladies. #awesometobeawoman
10. Good morning #Helsinki! Sun will rise in 30 minutes (local time 07:28)

Observação: emojis e alguns símbolos (°, setas) foram simplificados nesta versão em PDF; consulte a versão HTML para os tweets originais — lembrando que emojis costumam carregar boa parte do sentimento.

Anote suas classificações (de 1 a 10). O colega faz o mesmo, de forma independente — assim teremos **dois anotadores** para comparar.

2.2 Passo 2 — Calcular a concordância percentual

Queremos que os dados de treino sejam **confiáveis**, ou seja, que haja concordância sobre o que descrevemos. Uma forma de medir é a **concordância percentual**: quantas vezes, das 10, os dois anotadores concordaram, dividido pelo número de itens (10):

Python

```
# Substitua o numero abaixo pelo numero de itens em que voces concordaram
agreement = 0

# Divide a contagem pelo numero de tweets
agreement = agreement / 10

# Imprime a variavel
agreement
```

Saída

0.0

2.3 Passo 3 — Calcular as probabilidades de cada categoria

A concordância percentual é uma medida **muito ruim**: qualquer um pode ter **acertado por sorte** — ou classificado tudo aleatoriamente. A concordância percentual **não detecta** isso! Felizmente, dá para estimar a **concordância por acaso**. Primeiro, conte quantas vezes **você** usou cada categoria e converta em **probabilidades**, dividindo pelo total:

Python

```
# Conte quantos itens *voce* colocou em cada categoria
positive = 0
neutral = 0
negative = 0
```

```
# Converte as contagens em probabilidades
positive = positive / 10
neutral = neutral / 10
negative = negative / 10

# Chama cada variavel para examinar a saida
positive, neutral, negative
```

Saída

```
(0.0, 0.0, 0.0)
```

Essas probabilidades representam a chance de **você** escolher aquela categoria. Peça ao colega as probabilidades dele e registre-as:

Python

```
nb_positive = 0
nb_neutral = 0
nb_negative = 0
```

Com as probabilidades dos **dois** anotadores, calculamos a probabilidade de ambos escolherem a mesma categoria **por acaso**: para cada categoria, multiplicamos a sua probabilidade pela do colega:

Python

```
both_positive = positive * nb_positive
both_neutral = neutral * nb_neutral
both_negative = negative * nb_negative
```

Nota

Se um anotador não colocou nenhum *tweet* em uma categoria (ex.: *negative*) e o outro colocou, isso **anula** a chance de concordância por acaso naquela categoria — multiplicar por zero resulta em zero.

2.4 Passo 4 — Estimar a concordância esperada (por acaso)

Agora calculamos quão provável é concordar **por acaso**: a **concordância esperada** (*expected agreement*), somando as probabilidades combinadas de cada categoria:

Python

```
expected_agreement = both_positive + both_neutral + both_negative

expected_agreement
```

Saída

0.0

Com a **concordância observada** (`agreement`) e a **esperada por acaso** (`expected_agreement`), usamos uma medida mais confiável: o **kappa de Cohen** (κ), definido pela fórmula:

$$\kappa = \frac{P_{\text{observado}} - P_{\text{esperado}}}{1 - P_{\text{esperado}}}$$

Como a informação já está em `agreement` e `expected_agreement`, calculamos κ facilmente — note os **parênteses** para executar as subtrações antes da divisão:

Python

```
kappa = (agreement - expected_agreement) / (1 - expected_agreement)

kappa
```

Saída

0.0

3. O kappa de Cohen como medida de concordância

O valor teórico do κ vai de -1 (discordância perfeita) a $+1$ (concordância perfeita), com 0 indicando concordância totalmente aleatória. O κ costuma ser interpretado como a **força** da concordância. Landis e Koch (1977) propuseram os *benchmarks* abaixo — que devem ser levados **com cautela**, pois as divisões são arbitrárias:

Kappa de Cohen (κ)	Força da concordância
$\kappa < 0,00$	Pobre
$0,00 - 0,20$	Ligeira
$0,21 - 0,40$	Razoável
$0,41 - 0,60$	Moderada
$0,61 - 0,80$	Substancial
$0,81 - 1,00$	Quase perfeita

O κ serve para **dois** anotadores, com categorias fixadas de antemão. Para mais de dois, usa-se o κ **de Fleiss**. Na prática, ele já está implementado em bibliotecas — a **scikit-learn** (`sklearn`) tem a função `cohen_kappa_score()`, que recebe **duas listas**:

Python

```
# Importa a funcao cohen_kappa_score do modulo 'metrics' da scikit-learn
from sklearn.metrics import cohen_kappa_score

# Define duas listas, 'a1' e 'a2', com anotacoes de classes gramaticais
a1 = ['ADJ', 'AUX', 'NOUN', 'VERB', 'VERB']
```

```
a2 = ['ADJ', 'VERB', 'NOUN', 'NOUN', 'VERB']

# Usa cohen_kappa_score() para calcular a concordância entre as listas
cohen_kappa_score(a1, a2)
```

Saída

```
0.44444444444444453
```

Segundo o *benchmark* de Landis e Koch, esse valor indicaria concordância **moderada**.

Dica

Raramente é preciso anotar o conjunto inteiro — uma **amostra aleatória** costuma bastar. Se o κ sugere concordância, assumimos que as anotações são confiáveis (não aleatórias). Ainda assim, toda medida de concordância depende de suas próprias suposições — nenhuma representa a verdade absoluta.

4. Avaliando o desempenho de modelos de linguagem

Com um padrão-ouro confiável, podemos medir o desempenho de modelos. Suponha um padrão-ouro com 10 tokens anotados por classe gramatical (`gold_standard`) e as previsões de um modelo (`predictions`):

Python

```
# Define a lista 'gold_standard' (padrao-ouro anotado por humanos)
gold_standard = ['ADJ', 'ADJ', 'AUX', 'VERB', 'AUX', 'NOUN', 'NOUN', 'ADJ', 'DET',
                 'PRON']

# Define a lista 'predictions' (previsoes do modelo de linguagem)
predictions = ['NOUN', 'ADJ', 'AUX', 'VERB', 'AUX', 'NOUN', 'VERB', 'ADJ', 'DET', '
               PROPN']
```

4.1 Acurácia

A função `accuracy_score()` calcula a **acurácia**, que é exatamente a concordância observada calculada manualmente:

Python

```
# Importa o modulo 'metrics' da biblioteca scikit-learn (sklearn)
from sklearn import metrics

# Usa a funcao accuracy_score() do modulo 'metrics'
metrics.accuracy_score(gold_standard, predictions)
```

Saída

0.7

A acurácia sofre do mesmo problema — pode ser resultado de **palpites de sorte**. Como é um exemplo pequeno, dá para verificar que **7 de 10** classes coincidem: acurácia de 0,7 (70%).

4.2 Matriz de confusão

Para avaliar melhor, organizamos os resultados em uma **matriz de confusão**. Precisamos das classes únicas de ambas as listas, coletadas com `set()` — um **conjunto** (*set*) tem itens únicos, útil para remover duplicatas:

Python

```
# Coleta as classes gramaticais unicas em um conjunto, combinando as duas listas
pos_tags = set(gold_standard + predictions)

# Ordena o conjunto alfabeticamente e converte o resultado em lista
pos_tags = list(sorted(pos_tags))

# Imprime a lista resultante
pos_tags
```

Saída

```
['ADJ', 'AUX', 'DET', 'NOUN', 'PRON', 'PROPN', 'VERB']
```

Montamos uma tabela em que as **linhas** são o padrão-ouro e as **colunas** as previsões, somando **+1** na célula de cada par. O primeiro item de `gold_standard` é `ADJ` e o de `predictions` é `NOUN`:

Python

```
# Imprime o primeiro item de cada lista
gold_standard[0], predictions[0]
```

Saída

```
('ADJ', 'NOUN')
```

Somamos 1 na linha `ADJ`, coluna `NOUN`. A **matriz de confusão** completa (diagonal = acertos, em verde):

	ADJ	AUX	DET	NOUN	PRON	PROPN	VERB
ADJ	2	0	0	1	0	0	0
AUX	0	2	0	0	0	0	0
DET	0	0	1	0	0	0	0
NOUN	0	0	0	1	0	0	1
PRON	0	0	0	0	0	1	0
PROPN	0	0	0	0	0	0	0
VERB	0	0	0	0	0	0	1

As previsões corretas formam uma **diagonal** aproximada. Dela derivamos duas métricas por classe:

- **Precisão** (*precision*): proporção de previsões corretas **por classe**. A coluna **VERB** soma 2, mas só 1 está certa (na linha **VERB**): precisão de **VERB** = $1/2 = 0,5$. Idem para **NOUN**.
- **Revocação** (*recall*): proporção de acertos dentre **todos os exemplos reais** da classe. A linha **ADJ** soma 3, mas só 2 estão na coluna **ADJ**: revocação de **ADJ** = $2/3 \approx 0,66$. Para **NOUN**, $1/2 = 0,5$.

A scikit-learn gera a matriz automaticamente com `confusion_matrix()`:

Python

```
# Calcula a matriz de confusao para as duas listas e imprime o resultado
print(metrics.confusion_matrix(gold_standard, predictions))
```

Saída

```
[[2 0 0 1 0 0 0]
 [0 2 0 0 0 0 0]
 [0 0 1 0 0 0 0]
 [0 0 0 1 0 0 1]
 [0 0 0 0 0 1 0]
 [0 0 0 0 0 0 0]
 [0 0 0 0 0 0 1]]
```

4.3 Precisão e revocação com a scikit-learn

A **precisão** é implementada em `precision_score()`. Como há **mais de duas** classes, definimos como combinar os resultados pelo argumento `average`. Com `average=None`, calcula a precisão **de cada classe**. O `zero_division=0` evita erro quando uma classe de `predictions` não existe em `gold_standard`:

Python

```
# Calcula a precisao entre as duas listas, para cada classe (classe gramatical)
precision = metrics.precision_score(gold_standard, predictions, average=None,
                                   zero_division=0)

# Chama a variavel para examinar o resultado
precision
```

Saída

```
array([1. , 1. , 1. , 0.5, 0. , 0. , 0.5])
```

A saída é um **array NumPy**. Para combinar os rótulos de `pos_tags` com as pontuações, usamos `zip()` e convertemos com `dict()`:

Python

```
# Combina o conjunto 'pos_tags' com o array 'precision' usando zip();  
# converte o resultado em um dicionário  
dict(zip(pos_tags, precision))
```

Saída

```
{'ADJ': 1.0, 'AUX': 1.0, 'DET': 1.0, 'NOUN': 0.5, 'PRON': 0.0, 'PROPN': 0.0, 'VERB': 0.5}
```

Para uma **única** precisão de todas as classes, usamos `average='macro'`, que trata cada classe como igualmente importante:

Python

```
# Calcula a precisão entre as duas listas e tira a média (macro)  
macro_precision = metrics.precision_score(gold_standard, predictions, average='macro', zero_division=0)  
  
# Chama a variável para examinar o resultado  
macro_precision
```

Saída

```
0.5714285714285714
```

A precisão **macro** é a soma das precisões dividida pelo número de classes — dá para verificar manualmente:

Python

```
# Calcula a média macro manualmente: soma as precisões e divide  
# pelo número de classes em 'precision'  
sum(precision) / len(precision)
```

Saída

```
0.5714285714285714
```

A **revocação** é calculada do mesmo modo, com `recall_score()`:

Python

```
# Calcula a revocacao entre as duas listas, para cada classe
recall = metrics.recall_score(gold_standard, predictions, average=None,
                              zero_division=0)

# Combina 'pos_tags' com o array 'recall' e converte em dicionario
dict(zip(pos_tags, recall))
```

Saída

```
{'ADJ': 0.6666666666666666, 'AUX': 1.0, 'DET': 1.0, 'NOUN': 0.5, 'PRON': 0.0, '
  PROPN': 0.0, 'VERB': 1.0}
```

4.4 Relatório de classificação

A função `classification_report()` dá precisão e revocação de cada classe, com o **F1-score** — média equilibrada entre precisão e revocação (ambas contribuem igualmente), de 0 a 1:

Python

```
# Imprime um relatorio de classificacao
print(metrics.classification_report(gold_standard, predictions, zero_division=0))
```

Saída

	precision	recall	f1-score	support
ADJ	1.00	0.67	0.80	3
AUX	1.00	1.00	1.00	2
DET	1.00	1.00	1.00	1
NOUN	0.50	0.50	0.50	2
PRON	0.00	0.00	0.00	1
PROPN	0.00	0.00	0.00	0
VERB	0.50	1.00	0.67	1
accuracy			0.70	10
macro avg	0.57	0.60	0.57	10
weighted avg	0.75	0.70	0.71	10

As pontuações da linha `macro avg` correspondem às que calculamos acima. A linha `weighted avg` (média **ponderada**) leva em conta o número de instâncias de cada classe; a coluna `support` conta quantas instâncias foram observadas em cada classe.

Resumo da unidade

Nesta unidade, você aprendeu a

1. entender o que é um **padrão-ouro** e por que ele é uma **abstração** do uso da língua;
2. medir a **confiabilidade** entre anotadores: concordância percentual, probabilidades por categoria e concordância esperada por acaso;
3. calcular e interpretar o **kappa de Cohen** (manualmente e com `cohen_kappa_score()`), com os *benchmarks* de Landis & Koch;
4. avaliar modelos com **acurácia**, **matriz de confusão**, **precisão**, **revocação** e **F1-score** (`accuracy_score`, `confusion_matrix`, `precision_score`, `recall_score`, `classification_report`).

Exercícios sugeridos

1. Anote você mesmo os 10 *tweets* e peça a um colega que faça o mesmo; calcule a concordância percentual e o κ de Cohen com `cohen_kappa_score()`.
2. Monte suas próprias listas `gold_standard` e `predictions` (com pelo menos 8 itens) e gere a matriz com `confusion_matrix()`.
3. A partir do relatório, explique por que a classe `PROPN` tem `support` igual a 0 e o que isso significa.
4. Calcule o F1-score de `VERB` **manualmente** ($F1 = 2 \cdot (P \cdot R) / (P + R)$) e compare com o relatório.

Referências. *Applied Language Technology* (MOOC), Universidade de Helsinque — applied-language-technology.mooc.fi.
Landis, J. R.; Koch, G. G. (1977). *The measurement of observer agreement for categorical data*. Biometrics. Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR — scikit-learn.org.

Material produzido para o Programa CiberExt 26-29 / FEELT38103 — Universidade Federal de Uberlândia.